

Android 多权限申请及设置引导

Android 多权限同时申请 + 引导用户去设置开启权限详解

在实际开发中，我们常遇到需要同时申请多个危险权限（如拍照+存储），或用户拒绝权限并勾选“不再询问”后，需要引导用户去系统设置手动开启权限的场景。本文基于之前的 Lazy 模式，进一步讲解这两个进阶功能的实现（Java + XML）。

一、多权限同时申请（以“相机+存储”为例）

1. 核心逻辑

- 申请时将多个权限放入字符串数组，一次性提交给系统；
- 处理结果时遍历权限数组，检查每个权限的授予状态；
- 只有所有权限都被授予，才能执行后续操作。

2. 实战实现

步骤 1：清单文件声明多个权限

Code block

```
1  <!-- 相机权限 -->
2  <uses-permission android:name="android.permission.CAMERA" />
3  <!-- 存储权限（兼容安卓9及以下） -->
4  <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"
5                      android:maxSdkVersion="28" />
```

步骤 2：Java 代码实现多权限申请

修改 MainActivity.java，实现“拍照+存储”权限的同时申请：

Code block

```
1  import android.Manifest;
2  import android.content.Intent;
3  import android.content.pm.PackageManager;
4  import android.net.Uri;
5  import android.os.Build;
6  import android.os.Bundle;
7  import android.provider.Settings;
8  import android.view.View;
```

```
9  import android.widget.Button;
10 import android.widget.Toast;
11
12 import androidx.annotation.NonNull;
13 import androidx.appcompat.app.AppCompatActivity;
14 import androidx.core.app.ActivityCompat;
15 import androidx.core.content.ContextCompat;
16
17 public class MainActivity extends AppCompatActivity {
18
19     // 多权限请求码 (自定义)
20     private static final int REQUEST_CAMERA_STORAGE_PERMISSION = 1002;
21     private Button btnTakePhoto;
22
23     @Override
24     protected void onCreate(Bundle savedInstanceState) {
25         super.onCreate(savedInstanceState);
26         setContentView(R.layout.activity_main);
27
28         btnTakePhoto = findViewById(R.id.btn_take_photo);
29         btnTakePhoto.setOnClickListener(v -> checkMultiPermissions());
30     }
31
32     // 检查多个权限 (相机+存储)
33     private void checkMultiPermissions() {
34         // 1. 定义需要申请的权限数组
35         String[] requiredPermissions = {
36             Manifest.permission.CAMERA,
37             Manifest.permission.WRITE_EXTERNAL_STORAGE
38         };
39
40         // 2. 检查是否所有权限都已授予
41         boolean allGranted = true;
42         for (String permission : requiredPermissions) {
43             if (ContextCompat.checkSelfPermission(this, permission) !=
44                 PackageManager.PERMISSION_GRANTED) {
45                 allGranted = false;
46                 break;
47             }
48
49             if (allGranted) {
50                 // 所有权限已授予 → 执行拍照+保存操作
51                 takePhotoAndSave();
52             } else {
53                 // 存在未授予的权限 → 申请所有需要的权限
54                 requestMultiPermissions(requiredPermissions);
55             }
56         }
57     }
58 }
```

```

55         }
56     }
57
58     // 申请多个权限
59     private void requestMultiPermissions(String[] permissions) {
60         // 检查是否需要向用户解释权限用途
61         boolean needExplain = false;
62         for (String permission : permissions) {
63             if (ActivityCompat.shouldShowRequestPermissionRationale(this,
64                 permission)) {
65                 needExplain = true;
66                 break;
67             }
68         }
69         if (needExplain) {
70             Toast.makeText(this, "需要相机和存储权限才能拍照并保存照片",
71                 Toast.LENGTH_SHORT).show();
72         }
73
74         // 一次性申请多个权限
75         ActivityCompat.requestPermissions(
76             this,
77             permissions,
78             REQUEST_CAMERA_STORAGE_PERMISSION
79         );
80
81         // 处理多权限申请结果
82         @Override
83         public void onRequestPermissionsResult(
84             int requestCode,
85             @NonNull String[] permissions,

```

3. 关键细节

- **权限数组遍历**：检查和申请多个权限时，需遍历数组确认每个权限的状态；
- **结果校验**：只有所有权限都被授予，才能执行依赖这些权限的操作；
- **永久拒绝判断**：`shouldShowRequestPermissionRationale` 返回 `false` + 权限被拒绝 → 用户勾选了“不再询问”，此时只能引导去设置。

二、引导用户去设置开启权限（处理“永久拒绝”）

1. 核心场景

当用户拒绝权限并勾选“不再询问”后，后续调用 `requestPermissions` 不会再弹出系统弹窗，此时需要引导用户手动去系统设置开启权限。

2. 实现步骤

步骤 1：判断是否永久拒绝

通过 `shouldShowRequestPermissionRationale` 方法判断：

- **返回 true**：用户第一次拒绝，可再次申请并解释用途；
- **返回 false**：用户勾选了“不再询问”（永久拒绝），需引导去设置。

步骤 2：跳转到系统设置页面

使用 `Settings.ACTION_APPLICATION_DETAILS_SETTINGS` Intent 跳转：

Code block

```
1 // 跳转到当前App的权限设置页面
2 private void goToAppSettings() {
3     Intent intent = new Intent(Settings.ACTION_APPLICATION_DETAILS_SETTINGS);
4     Uri uri = Uri.fromParts("package", getPackageName(), null);
5     intent.setData(uri);
6     startActivity(intent);
7 }
```

步骤 3：优化体验（自定义弹窗提示）

直接跳转可能让用户困惑，建议先弹出自定义 Dialog 说明原因：

Code block

```
1 import android.app.AlertDialog;
2
3 // 自定义弹窗引导去设置
4 private void showPermissionSettingDialog() {
5     new AlertDialog.Builder(this)
6         .setTitle("权限需要手动开启")
7         .setMessage("相机和存储权限已被永久拒绝，无法拍照和保存照片，请前往设置开启权限")
8         .setPositiveButton("去设置", (dialog, which) -> goToAppSettings())
9         .setNegativeButton("取消", null)
10        .show();
11 }
```

3. 注意事项

- **跳转后的处理：**用户从设置返回后，可在 `onResume` 方法中重新检查权限状态，若已开启则执行操作；
- **避免频繁跳转：**不要在用户每次操作时都强制跳转设置，需友好提示。

三、完整流程总结（多权限+设置引导）

1. **用户触发操作**（如点击拍照按钮）；
2. **检查多个权限：**遍历权限数组，确认是否全部授予；
3. **申请权限：**未授予则一次性申请所有需要的权限；
4. **处理结果：**
 - 全部授予 → 执行操作；
 - 部分/全部拒绝 → 检查是否永久拒绝；
5. **永久拒绝处理：**弹出提示并引导用户去系统设置开启权限；
6. **用户返回后重新检查：**若权限已开启，执行操作；否则提示功能不可用。

四、兼容性与优化建议

1. 安卓版本适配

- **安卓 10+ (API 29)：**存储权限 `WRITE_EXTERNAL_STORAGE` 失效，无需申请，使用分区存储（MediaStore）保存文件；
- **安卓 13+ (API 33)：**存储权限拆分为 `READ_MEDIA_IMAGES`、`READ_MEDIA_VIDEO` 等，需适配新权限。

2. 代码封装（权限工具类）

将多权限检查、申请、设置引导逻辑封装为工具类，减少重复代码：

Code block

```
1  import android.app.Activity;
2  import android.content.Intent;
3  import android.content.pm.PackageManager;
4  import android.net.Uri;
5  import android.provider.Settings;
6
7  import androidx.core.app.ActivityCompat;
8  import androidx.core.content.ContextCompat;
9
10 import java.util.ArrayList;
11 import java.util.List;
12
```

```
13 public class PermissionManager {
14
15     // 检查多个权限是否已授予
16     public static boolean checkAllPermissions(Activity activity, String[]
permissions) {
17         for (String permission : permissions) {
18             if (ContextCompat.checkSelfPermission(activity, permission) !=
PackageManager.PERMISSION_GRANTED) {
19                 return false;
20             }
21         }
22         return true;
23     }
24
25     // 获取未授予的权限列表
26     public static List<String> getDeniedPermissions(Activity activity,
String[] permissions) {
27         List<String> deniedList = new ArrayList<>();
28         for (String permission : permissions) {
29             if (ContextCompat.checkSelfPermission(activity, permission) !=
PackageManager.PERMISSION_GRANTED) {
30                 deniedList.add(permission);
31             }
32         }
33         return deniedList;
34     }
35
36     // 申请权限
37     public static void requestPermissions(Activity activity, String[]
permissions, int requestCode) {
38         ActivityCompat.requestPermissions(activity, permissions, requestCode);
39     }
40
41     // 判断是否有永久拒绝的权限
42     public static boolean hasPermanentDenied(Activity activity, String[]
permissions) {
43         for (String permission : permissions) {
44             if (ContextCompat.checkSelfPermission(activity, permission) !=
PackageManager.PERMISSION_GRANTED
45                 &&
!ActivityCompat.shouldShowRequestPermissionRationale(activity, permission)) {
46                 return true;
47             }
48         }
49         return false;
50     }
51 }
```

```

52      // 跳转到App设置页面
53      public static void goToSettings(Activity activity) {
54          Intent intent = new
Intent(Settings.ACTION_APPLICATION_DETAILS_SETTINGS);
55          Uri uri = Uri.fromParts("package", activity.getPackageName(), null);
56          intent.setData(uri);
57          activity.startActivity(intent);
58      }
59  }

```

使用工具类简化代码：

Code block

```

1  // 检查权限
2  if (PermissionManager.checkAllPermissions(this, requiredPermissions)) {
3      takePhotoAndSave();
4  } else {
5      // 获取未授予的权限列表
6      List<String> deniedPermissions =
PermissionManager.getDeniedPermissions(this, requiredPermissions);
7      PermissionManager.requestPermissions(this, deniedPermissions.toArray(new
String[0]), REQUEST_CODE);
8  }
9
10 // 检查永久拒绝
11 if (PermissionManager.hasPermanentDenied(this, requiredPermissions)) {
12     PermissionManager.goToSettings(this);
13 }

```

五、常见问题排查

1. 跳转设置后权限已开启，但返回App后仍提示无权限？

- 需在 `onResume` 方法中重新检查权限状态：

Code block

```

1  @Override
2  protected void onResume() {
3      super.onResume();
4      // 重新检查权限
5      String[] requiredPermissions = {Manifest.permission.CAMERA,
Manifest.permission.WRITE_EXTERNAL_STORAGE};
6      if (PermissionManager.checkAllPermissions(this, requiredPermissions)) {
7          takePhotoAndSave();

```

```
8     }  
9 }
```

2. 安卓13+申请存储权限无反应？

- 安卓13+将存储权限拆分为 `READ_MEDIA_IMAGES`、`READ_MEDIA_VIDEO` 等，需替换旧的存储权限：

Code block

```
1  String[] requiredPermissions = {  
2      Manifest.permission.CAMERA,  
3      Manifest.permission.READ_MEDIA_IMAGES // 安卓13+存储图片权限  
4  };
```

3. 多权限申请时部分拒绝，如何处理？

- 可针对已授予的权限执行部分操作，或提示用户“缺少XX权限，功能受限”。

通过以上实现，你可以优雅地处理多权限申请和永久拒绝场景，符合 Android 权限设计规范，提升用户体验。如果需要进一步优化（如权限申请结果的 LiveData 监听），可以结合 Jetpack 组件实现更灵活的权限管理！

（注：文档部分内容可能由 AI 生成）